

Course Outline: Introducción al Testing

Title: Introducción al Testing de Software **Learnpack:** + Introducción al testing **Prerequisite:** Fundamentos de Python **Target Audience:** Beginner developers learning software testing fundamentals **Total Lessons:** 10 **Format:** Mixed (40% hands-on)

Course Description

This course introduces students to the fundamental concepts of software testing, focusing on why testing matters, the main types of testing, and Test-Driven Development (TDD). Students will learn to write unit tests using Python and pytest, understand when and how to test their code, and develop the mindset of quality-first development. The course emphasizes practical application through hands-on coding exercises that demonstrate testing concepts in real scenarios.

By the end of this course, students will understand the value proposition of testing, be able to write effective unit tests, and know how to apply TDD principles to build more reliable software from the ground up.

Course Philosophy

This course emphasizes:

- **Quality-first mindset:** Understanding that testing saves time and prevents bugs rather than slowing development
- **Practical application:** Learning testing through real code examples and challenges students can immediately use
- **TDD as a design tool:** Using tests not just for verification but as a way to think through code design
- **Human-in-the-loop:** Balancing automated testing with critical thinking about what and when to test

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Why Testing Matters	2
02 - Types of Testing	3
03 - Test-Driven Development	2
04 - Assessment and Mastery	2
05 - Outro	1
Total	10

Section 00: Welcome

00.0 Welcome to Software Testing

Learning Objectives:

- Understand what software testing is and why it's essential for developers
- Recognize the connection between testing and code quality
- Preview the testing concepts and skills covered in this course

Content Outline:

- The reality of bugs in software and their impact
- What testing means in the context of software development
- Overview of testing types: unit, functional, integration, performance
- Introduction to Test-Driven Development as a development philosophy
- Course journey: from understanding why to implementing how

Transitions: This lesson sets the foundation for understanding why testing exists. Next, we'll explore the concrete costs of bugs and the value testing provides to developers and users.

Key Principles:

- Testing is an investment in code quality and developer confidence
- Different types of testing serve different purposes in software development
- Testing is not just about finding bugs—it's about preventing them
- Good testing practices make development faster, not slower

Examples:

- A calculator app that works for addition but fails for division by zero
 - An e-commerce system that processes payments but crashes during high traffic
 - A login system that accepts valid credentials but fails with special characters in passwords
-

Section 01: Why Testing Matters

01.0 The Cost of Bugs and Value of Testing

Learning Objectives:

- Calculate the real cost of bugs in terms of time, money, and user experience
- Understand how testing prevents bugs rather than just finding them
- Recognize testing as a tool for developer confidence and faster development

Content Outline:

- The exponential cost of fixing bugs: development vs production
- Real-world examples of expensive software failures
- How testing acts as a safety net for code changes
- Testing as documentation: showing how code is supposed to work
- The feedback loop: faster detection leads to easier fixes
- Testing's role in enabling confident refactoring and feature additions

Transitions: Now that we understand why testing is valuable, let's get hands-on experience by writing our first unit test to see testing in action.

Key Principles:

- Bug costs increase exponentially the later they're discovered
- Testing enables confident code changes and refactoring
- Tests serve as living documentation of how code should behave
- Early feedback through testing accelerates development

Examples:

- A bug caught during development takes 1 hour to fix; the same bug in production takes 10+ hours
- Knight Capital's \$440 million loss due to untested code deployment
- How adding tests to a payment processing function prevents accidental changes from breaking existing functionality

Anti-patterns (woven in):

- **Testing only happy paths:** Only testing when everything works perfectly, ignoring error cases and edge conditions that commonly cause real-world failures
 - **Testing too late:** Writing tests after code is complete and deployed, missing the opportunity to catch design issues early
-

01.1 Writing Your First Unit Test

Challenge Prompt: Create a simple calculator function and write unit tests for it using pytest to verify both normal operations and edge cases.

Solution Code:

```
import pytest

def calculator(a, b, operation):
    if operation == "add":
        return a + b
    elif operation == "subtract":
        return a - b
    elif operation == "multiply":
        return a * b
    elif operation == "divide":
        if b == 0:
            raise ValueError("Cannot divide by zero")
        return a / b
    else:
        raise ValueError("Invalid operation")

def test_addition():
    assert calculator(5, 3, "add") == 8

def test_subtraction():
    assert calculator(5, 3, "subtract") == 2

def test_multiplication():
    assert calculator(5, 3, "multiply") == 15

def test_division():
    assert calculator(6, 3, "divide") == 2

def test_division_by_zero():
    with pytest.raises(ValueError):
        calculator(5, 0, "divide")

def test_invalid_operation():
    with pytest.raises(ValueError):
        calculator(5, 3, "invalid")
```

Challenge Code:

```
import pytest

def calculator(a, b, operation):
    # your code here
    pass

def test_addition():
    # your code here
```

```
pass

def test_subtraction():
    # your code here
    pass

def test_multiplication():
    # your code here
    pass

def test_division():
    # your code here
    pass

def test_division_by_zero():
    # your code here
    pass

def test_invalid_operation():
    # your code here
    pass
```

Section 02: Types of Testing

02.0 Unit Tests vs Integration vs Functional vs Performance Testing 📖

Learning Objectives:

- Distinguish between unit, integration, functional, and performance testing
- Understand when to use each type of testing in development
- Recognize unit testing as the foundation of a comprehensive testing strategy
- Identify the scope and purpose of each testing type

Content Outline:

- **Unit Testing:** Testing individual functions or methods in isolation
 - Focus: Does this one piece of code work correctly?
 - Speed: Fast execution, immediate feedback
 - Scope: Single function, single responsibility
- **Integration Testing:** Testing how different parts of the system work together
 - Focus: Do these components communicate correctly?
 - Examples: API calls, database connections, file operations
- **Functional Testing:** Testing complete features from a user perspective
 - Focus: Does the feature work as users expect?
 - Examples: User can login, place an order, upload a file
- **Performance Testing:** Testing system behavior under load
 - Focus: Does the system perform adequately under stress?
 - Examples: Response times, concurrent users, memory usage

Transitions: Understanding these testing types helps us choose the right approach for different scenarios. Next, we'll learn how to identify test cases and edge cases that make our unit tests comprehensive.

Key Principles:

- Unit tests form the foundation—they're fast, focused, and catch issues early
- Each testing type serves a different purpose and provides different confidence levels
- Most applications need primarily unit tests with selective integration and functional tests

- Testing strategy should match application architecture and risk areas

Examples:

- Unit test: Testing a `validate_email()` function with various email formats
 - Integration test: Testing that user registration saves to database and sends welcome email
 - Functional test: Testing complete user signup flow from form submission to account activation
-

02.1 Identifying Test Cases and Edge Cases

Learning Objectives:

- Identify normal use cases that should be tested
- Recognize common edge cases and boundary conditions
- Develop systematic thinking about what can go wrong
- Apply equivalence partitioning to reduce test redundancy

Content Outline:

- **Normal cases:** Expected inputs and typical user behavior
- **Edge cases:** Boundary values and unusual but valid inputs
 - Empty strings, zero values, maximum/minimum limits
 - Special characters, very large numbers, null values
- **Error cases:** Invalid inputs and system failures
 - Wrong data types, network failures, permission issues
- **Equivalence partitioning:** Grouping similar test cases
 - Testing one representative from each group instead of every possible input
- **Boundary value analysis:** Testing at the edges of valid ranges
- **Error path testing:** Ensuring graceful handling of failures

Transitions: Now that we know how to identify comprehensive test cases, let's build a complete test suite that covers normal cases, edge cases, and error scenarios.

Key Principles:

- Think about what can go wrong, not just what should go right
- Boundary conditions often reveal bugs that normal cases miss
- Test one representative from each equivalence class
- Error cases are as important as success cases

Examples:

- Testing a password validator: normal passwords, too short, too long, special characters, empty string
- Testing an age calculator: valid dates, future dates, leap years, invalid formats
- Testing a shopping cart: adding items, removing items, empty cart, exceeding inventory

Anti-patterns (woven in):

- **Over-testing similar cases:** Writing 50 tests for different valid email formats instead of testing one from each category (valid format, invalid format, edge cases)
 - **Ignoring error paths:** Only testing when functions succeed, not when they should fail gracefully
-

02.2 Building a Complete Test Suite

Challenge Prompt: Build a comprehensive test suite for a user validation system that covers normal cases, edge cases, and error handling scenarios.

Solution Code:

```

import pytest

def validate_user(name, age, email):
    if not isinstance(name, str) or len(name.strip()) == 0:
        raise ValueError("Name must be a non-empty string")
    if not isinstance(age, int) or age < 0 or age > 150:
        raise ValueError("Age must be between 0 and 150")
    if not isinstance(email, str) or "@" not in email or "." not in email:
        raise ValueError("Email must contain @ and .")
    return {"name": name.strip(), "age": age, "email": email.lower()}

def test_valid_user():
    result = validate_user("Alice", 25, "alice@example.com")
    assert result == {"name": "Alice", "age": 25, "email": "alice@example.com"}

def test_name_with_whitespace():
    result = validate_user(" Bob ", 30, "bob@test.com")
    assert result["name"] == "Bob"

def test_email_case_normalization():
    result = validate_user("Carol", 22, "CAROL@EXAMPLE.COM")
    assert result["email"] == "carol@example.com"

def test_boundary_ages():
    validate_user("Young", 0, "young@test.com") # Should pass
    validate_user("Old", 150, "old@test.com") # Should pass

def test_empty_name():
    with pytest.raises(ValueError, match="Name must be"):
        validate_user("", 25, "test@example.com")

def test_invalid_age_negative():
    with pytest.raises(ValueError, match="Age must be"):
        validate_user("Alice", -1, "alice@example.com")

def test_invalid_age_too_high():
    with pytest.raises(ValueError, match="Age must be"):
        validate_user("Alice", 151, "alice@example.com")

def test_invalid_email_no_at():
    with pytest.raises(ValueError, match="Email must contain"):
        validate_user("Alice", 25, "aliceexample.com")

```

Challenge Code:

```

import pytest

def validate_user(name, age, email):
    # your code here
    pass

def test_valid_user():
    # your code here
    pass

def test_name_with_whitespace():
    # your code here
    pass

```

```
def test_email_case_normalization():
    # your code here
    pass

def test_boundary_ages():
    # your code here
    pass

def test_empty_name():
    # your code here
    pass

def test_invalid_age_negative():
    # your code here
    pass

def test_invalid_age_too_high():
    # your code here
    pass

def test_invalid_email_no_at():
    # your code here
    pass
```

Section 03: Test-Driven Development

03.0 The TDD Cycle: Red, Green, Refactor

Learning Objectives:

- Understand the three phases of TDD: Red, Green, Refactor
- Recognize how TDD changes the development approach from code-first to test-first
- Appreciate TDD as a design tool that leads to better code architecture
- Learn when TDD is most beneficial and when it might be overkill

Content Outline:

- **The TDD cycle explained:**
 - **Red:** Write a failing test for the desired functionality
 - **Green:** Write just enough code to make the test pass
 - **Refactor:** Improve code quality while keeping tests passing
- **TDD as design methodology:** Tests help clarify requirements before coding
- **Benefits of TDD:**
 - Better code coverage by design
 - Simpler, more focused code (only what's needed)
 - Built-in regression testing
 - Living documentation of expected behavior
- **When to use TDD:** Complex logic, unclear requirements, critical functionality
- **When TDD might be overkill:** Simple CRUD operations, exploratory coding, tight deadlines
- **TDD vs. traditional testing:** Writing tests first vs. after implementation

Transitions: Understanding the TDD philosophy prepares us for hands-on practice. Next, we'll implement a complete feature using the Red-Green-Refactor cycle.

Key Principles:

- Write the simplest test that fails, then the simplest code that passes
- Refactor only when tests are passing to ensure no functionality is lost
- TDD drives better design by forcing you to think about interfaces first
- Small cycles keep development focused and reduce debugging time

Examples:

- TDD for a password strength checker: test weak password → implement basic check → refactor for multiple criteria
- TDD for a shopping cart: test adding item → implement add method → refactor for duplicate item handling
- TDD for user authentication: test valid login → implement basic auth → refactor for security measures

Anti-patterns (woven in):

- **Writing too many tests at once:** Creating multiple failing tests before implementing any functionality, violating the "one failing test" rule and making it hard to focus
- **Skipping the refactor step:** Writing code that passes tests but never improving its structure, leading to technical debt even with good test coverage

03.1 TDD in Practice: Building Features Test-First

Challenge Prompt: Use the TDD approach to build a task manager that can add tasks, mark them complete, and list pending tasks, following the Red-Green-Refactor cycle.

Solution Code:

```
import pytest

class TaskManager:
    def __init__(self):
        self.tasks = []

    def add_task(self, description):
        if not description or not description.strip():
            raise ValueError("Task description cannot be empty")
        task = {"id": len(self.tasks) + 1, "description": description.strip(), "completed": False}
        self.tasks.append(task)
        return task["id"]

    def complete_task(self, task_id):
        task = self._find_task(task_id)
        if task:
            task["completed"] = True
            return True
        return False

    def get_pending_tasks(self):
        return [task for task in self.tasks if not task["completed"]]

    def _find_task(self, task_id):
        return next((task for task in self.tasks if task["id"] == task_id), None)

# Tests written first in TDD approach
def test_add_task():
    tm = TaskManager()
    task_id = tm.add_task("Write tests")
    assert task_id == 1
    assert len(tm.tasks) == 1
```



```

def test_complete_task():
    tm = TaskManager()
    task_id = tm.add_task("Complete task")
    result = tm.complete_task(task_id)
    assert result == True
    assert tm.tasks[0]["completed"] == True

def test_get_pending_tasks():
    tm = TaskManager()
    tm.add_task("Pending task")
    tm.add_task("Another pending")
    task_id = tm.add_task("To complete")
    tm.complete_task(task_id)

    pending = tm.get_pending_tasks()
    assert len(pending) == 2
    assert all(not task["completed"] for task in pending)

def test_empty_task_description():
    tm = TaskManager()
    with pytest.raises(ValueError):
        tm.add_task("")

```

Challenge Code:

```

import pytest

class TaskManager:
    def __init__(self):
        # your code here
        pass

    def add_task(self, description):
        # your code here - implement after writing test
        pass

    def complete_task(self, task_id):
        # your code here - implement after writing test
        pass

    def get_pending_tasks(self):
        # your code here - implement after writing test
        pass

# Write these tests FIRST, then implement methods above
def test_add_task():
    # your test code here
    pass

def test_complete_task():
    # your test code here
    pass

def test_get_pending_tasks():
    # your test code here
    pass

def test_empty_task_description():

```

```
# your test code here
pass
```

Section 04: Assessment and Mastery

04.0 Testing Best Practices and Anti-Patterns

Learning Objectives:

- Apply testing best practices to write maintainable and effective tests
- Identify and avoid common testing anti-patterns that reduce test value
- Understand the balance between thorough testing and practical development speed
- Recognize when to focus testing efforts for maximum impact

Content Outline:

- **Testing best practices:**
 - Test naming: descriptive names that explain the scenario
 - Test organization: grouping related tests, clear arrange-act-assert structure
 - Test independence: each test should run in isolation
 - Test data management: using fixtures and avoiding hardcoded values
 - Testing error conditions as thoroughly as success conditions
- **Common anti-patterns and their fixes:**
 - **Over-testing:** Testing every getter/setter vs. focusing on business logic
 - **Brittle tests:** Tests that break with minor code changes
 - **Slow tests:** Unit tests that hit databases or external services
 - **Testing implementation details:** Testing how vs. what the code does
- **Strategic testing:** Where to invest testing effort for maximum return
- **Test maintenance:** Keeping tests valuable as code evolves

Transitions: With these best practices in mind, you're ready to demonstrate your testing knowledge in the assessment that follows.

Key Principles:

- Good tests are readable, reliable, and focused on behavior not implementation
- Test failure should clearly indicate what went wrong and why
- Invest testing effort where bugs would be most costly
- Tests should make refactoring safer, not harder

Examples:

- Good test name: `test_user_login_with_invalid_password_returns_error` vs. bad: `test_login`
- Testing behavior: `assert user.is_authenticated()` vs. implementation: `assert user._auth_token is not None`
- Focused testing: Extensive tests for payment processing, basic tests for user profile display

Anti-patterns (comprehensive review):

- **Over-testing similar scenarios:** Writing separate tests for email "alice@test.com", "bob@test.com", "carol@test.com" instead of one test for valid email format
- **Testing only happy paths:** Comprehensive tests for successful user registration but no tests for duplicate email, invalid data, or server errors
- **Skipping error testing:** Testing that a function returns correct values but not testing what happens with invalid inputs or system failures

04.1 Testing Knowledge Assessment

Learning Objectives:

- Demonstrate understanding of testing fundamentals and best practices
- Apply testing concepts to realistic development scenarios
- Evaluate testing strategies for different types of applications
- Show mastery of TDD principles and implementation

Question Format: Multiple choice

Assessment Questions:

1. **What is the primary benefit of writing unit tests?** a) They make code run faster b) They catch bugs early when they're cheaper to fix c) They eliminate the need for manual testing d) They automatically generate documentation
2. **In the TDD cycle, what does the "Red" phase represent?** a) Writing code that passes existing tests b) Refactoring code to improve its structure c) Writing a failing test for desired functionality d) Running tests to see which ones fail
3. **Which scenario is BEST suited for unit testing?** a) Testing if the entire user registration flow works end-to-end b) Testing if a password validation function correctly identifies weak passwords c) Testing if the database connection is working d) Testing if the website loads quickly under high traffic
4. **What makes a test "brittle" and should be avoided?** a) The test runs slowly b) The test fails when implementation details change, even though behavior is correct c) The test checks error conditions d) The test uses realistic test data
5. **When testing a function that calculates shipping costs, which test cases are most important?** a) Only test with the most common shipping destinations b) Test normal cases, edge cases (zero weight, maximum distance), and error cases (invalid inputs) c) Test every possible combination of weight and distance d) Only test the mathematical calculation, not error handling
6. **What is a key anti-pattern to avoid when writing tests?** a) Testing error conditions in addition to success cases b) Writing descriptive test names that explain the scenario c) Writing separate tests for each type of invalid input d) Writing many tests that check slightly different variations of the same valid input
7. **In which situation would TDD be most beneficial?** a) Building a simple CRUD API with well-known requirements b) Implementing complex business logic with unclear edge cases c) Creating a quick prototype to test user interest d) Copying functionality from an existing, well-tested system

Evaluation Criteria:

- **7 correct:** Excellent understanding of testing fundamentals and practices
- **5-6 correct:** Good grasp with minor gaps in advanced concepts
- **3-4 correct:** Basic understanding, recommend reviewing TDD and best practices
- **Below 3:** Fundamental gaps, recommend reviewing entire course content

Score Interpretation: This assessment validates your understanding of why testing matters, how to implement effective tests, and when to apply different testing strategies. Strong performance indicates readiness to apply testing practices in real development projects.

Section 05: Outro

05.0 Your Testing Journey Forward

Content Outline:

- **Course recap:** You've learned why testing prevents costly bugs, how to write comprehensive unit tests covering normal and edge cases, and how to apply TDD to build better software from the ground up
- **Connection to bigger picture:** Testing skills integrate with everything you'll build—every API endpoint, frontend component, and data processing function benefits from good tests
- **Next steps and continued learning:**

- Practice TDD on your next project, even if it's small
- Explore pytest fixtures and parameterized tests for more advanced scenarios
- Learn about mocking and integration testing for complex applications
- Study testing strategies for web applications and APIs
- **Resources for further exploration:**
 - pytest documentation and plugins ecosystem
 - "Test Driven Development: By Example" by Kent Beck
 - Testing best practices for your specific tech stack
 - Open source projects with excellent test suites to study
- **Encouragement and celebration:** You now have a quality-first mindset that will make you a more confident and effective developer—every test you write is an investment in code that works reliably and can be changed safely

Note: This is conclusion only - no theory, exercises, or assessment.
